Current Project Report

Date: 2026-03-28

Project: Legal AI Platform

Executive Summary

The project is no longer just a backend prototype.

It now includes:

- a working backend for legal case and document management
- an internal staff-facing legal workspace frontend
- a separate client-facing intake portal
- voice intake and transcription support
- transcript-to-consultation extraction workflow
- a first real AI agent for summarization
- provider-agnostic LLM configuration for OpenAI-compatible APIs

The platform is now split into two product surfaces:

1. Internal Legal Workspace

Used by:

- lawyers
- admins
- assistants

Capabilities:

- authentication
- case creation and management
- client creation and management
- document upload and AI processing
- document intelligence review
- grounded legal copilot
- voice recording upload/review
- transcript-to-intake conversion

2. Client Intake Portal

Used by:

- external clients

Capabilities:

- submit consultation requests

- upload voice notes
- record voice notes in-browser
- upload supporting documents
- provide scheduling preferences
- check request status via public reference

Restricted from:

- internal models
- agent controls
- evidence panels
- internal case workspace operations

Current Architecture

The repository currently contains:

- ‘backend/‘ for API, business logic, AI services, and persistence
- ‘frontend/‘ for the internal legal workspace
- ‘client-portal/‘ for the public client-facing intake portal
- ‘docs/‘ for architecture and sprint reporting

Infrastructure currently expected by the backend:

- PostgreSQL
- Redis
- MinIO
- Docker

Backend Status

Core Platform

Implemented:

- user registration and login
- multi-tenant structure
- users, clients, cases, documents
- role-aware backend logic
- MinIO-based file storage
- Redis environment support

Key files:

- ‘backend/main.py‘
- ‘backend/api/auth.py‘
- ‘backend/api/cases.py‘
- ‘backend/api/clients.py‘
- ‘backend/core/config.py‘

Document Intelligence

Implemented:

- document upload
- text extraction
- normalization
- PII redaction
- chunking
- entity extraction
- FAISS indexing
- lexical + semantic retrieval
- full document analysis
- document summarization

Key files:

- ‘backend/services/ai/document_ai_pipeline.py‘
- ‘backend/services/ai/rag_service.py‘
- ‘backend/services/ai/summarization_service.py‘
- ‘backend/api/intelligence.py‘
- ‘backend/api/rag.py‘

Voice and Intake Workflow

Implemented:

- voice recording persistence
- voice upload API
- transcription service integration
- transcript storage
- transcript intake extraction
- consultation request persistence
- consultation request creation from transcript

Key files:

- ‘backend/models/voice_recording.py‘
- ‘backend/models/consultation_request.py‘
- ‘backend/api/voice.py‘
- ‘backend/api/consultations.py‘
- ‘backend/services/ai/transcription_service.py‘
- ‘backend/services/ai/transcript_intake_service.py‘

Public Client Intake

Implemented:

- public consultation submission endpoint
- public status lookup endpoint

- automatic case + client + consultation creation from public intake
- optional voice note processing
- optional supporting document upload

Key files:

- ‘backend/api/public.py‘
- ‘backend/api/public_schema.py‘

Frontend Status

Internal Workspace

Implemented in ‘frontend/‘:

- login and registration
- case list
- case detail workspace
- client creation
- case creation
- document upload
- document intelligence panel
- copilot chat panel
- evidence panel
- voice intake panel
- consultation intake review panel

Main files:

- ‘frontend/src/App.tsx‘
- ‘frontend/src/lib/api.ts‘
- ‘frontend/src/types.ts‘
- ‘frontend/src/styles.css‘

Client Portal

Implemented in ‘client-portal/‘:

- consultation submission form
- voice upload
- browser audio recording
- supporting document upload
- public status lookup by reference

Main files:

- ‘client-portal/src/App.tsx‘
- ‘client-portal/src/lib/api.ts‘
- ‘client-portal/src/types.ts‘
- ‘client-portal/src/styles.css‘

AI / Agent Status

Current AI Features

Implemented:

- heuristic document intelligence
- grounded RAG responses
- case-aware copilot behaviors
- transcript intake extraction

First Agent Implemented

The first explicit agent is now:

Summarization Agent

Purpose:

- improve document summaries using an LLM while keeping the deterministic heuristic pipeline as fallback

Implemented files:

- ‘backend/services/ai/agents/summarization_agent.py‘
- ‘backend/services/ai/llm_gateway.py‘

Provider-Agnostic LLM Layer

The project can now use an OpenAI-compatible provider instead of direct OpenAI-only assumptions.

Config added:

- ‘OPENAI_API_KEY‘
- ‘LLM_BASE_URL‘
- ‘LLM_MODEL‘
- ‘SUMMARY_AGENT_MODEL‘
- ‘OPENROUTER_SITE_URL‘
- ‘OPENROUTER_APP_NAME‘

This means the project is ready to use providers such as OpenRouter through the same OpenAI-compatible SDK path.

Sprint Status

Effectively Completed

- Sprint 1: Backend Stabilization

- Sprint 2: Internal Case Workspace v1
- Sprint 3: Voice Intake & Interaction v1
- Sprint 4: Transcript Intelligence / Intake Workflow

Partially Started

- Multi-Agent Legal AI

Evidence:

- first explicit agent created
- LLM provider abstraction added
- improved summarization path integrated

Not Yet Completed

- full multi-agent orchestration
- verifier agent / proof-first validation layer
- trainable ML models
- full case-level reasoning across documents and transcripts
- calendar/session booking integration
- production monitoring and test coverage maturity

What Works Right Now

A realistic current flow is:

1. staff user logs into the internal workspace
2. creates clients and cases
3. uploads legal documents
4. gets document intelligence and grounded copilot support
5. uploads or records client voice notes
6. transcribes those notes
7. converts transcript into consultation/intake data
8. reviews extracted booking and issue information

Separately:

1. a client uses the public portal
2. submits a consultation request
3. optionally records/uploads a voice note
4. optionally uploads a supporting file
5. receives a public reference
6. checks intake status later with that reference

Verification Status

Verified:

- backend compilation succeeded after the latest changes

- frontend source trees exist for both apps
- report artifacts and sprint documents exist

Not verified in this environment:

- ‘npm‘ frontend build/runtime, because Node.js is not installed in the current environment
- live provider-backed LLM calls, because they depend on local runtime/API access
- live browser recording flow end-to-end outside code inspection

Main Remaining Gaps

The biggest remaining gaps are:

- real multi-agent orchestration between specialized agents
- proof-first verifier layer
- stronger case-level reasoning over documents + transcripts together
- structured session/appointment scheduling integration
- stronger testing and production hardening

Recommended Next Step

The strongest next step is:

Build the next agent layer

Recommended order:

- Intake Agent
- Retrieval Agent
- Case Reasoning Agent
- Drafting Agent
- Verifier Agent

Why:

- the business workflows already exist
- the client/internal separation is already in place
- the first summarization agent and provider abstraction are already done

Final Assessment

The project is now at a meaningful product stage.

It already demonstrates:

- platform thinking
- role separation
- client/staff boundary design
- voice intake
- consultation extraction

- grounded legal AI
- first-step agent architecture

It is no longer just ?an API with AI utilities.?

It is now a multi-surface legal AI platform with internal operations, public intake, and the first step toward explicit agent-based architecture.